DIGITAL IMAGE PROCESSING TERM PROJECT 01

데니스 고레츠키, 이주환

Contents

Outline of Program

Purpose

The program is designed to perform basic image compositing operations, specifically addition and subtraction of images, while handling potential overflow and underflow in pixel values. This allows simple image blending and manipulation for educational purposes.

Functionality Overview

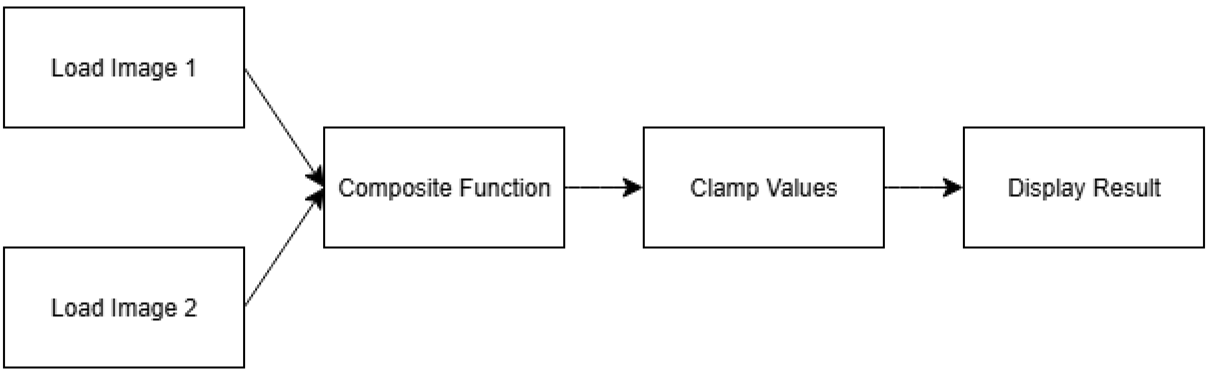
- Load input images using CxImage library
- Perform composite operations:
  - Pixel-wise addition of two images
  - Pixel-wise subtraction of one image from another
- Clamp pixel values to valid range (0–255 for 8-bit images)
- Display the resulting image

Design of Program

High-Level Design

- **Input Module**: Handles loading of images from disk
- **Composite Module**: Implements the addition and subtraction logic. Key steps include:
  1. Looping over each pixel of the input images
  2. Performing the addition or subtraction
  3. Clamping the result to avoid underflow/overflow
- **Output Module**: Displays the final image

Flow Diagram



Implementation Notes

- The Composite function is implemented in a single method using simple loops over image pixels
- The program is modular, allowing easy extension for additional composite operations in the future

Time spent

Planning & Design: ~30 minutes

Implementation of Composite Function: ~45 minutes

Testing & Debugging: ~30 minutes

Meetings & Collaboration: ~1 hour

Total Estimated Time: ~2.5 hours

Process of Project and Record of Project Meetings

Meeting 1 – Project Kickoff

- Discussed project requirements
- Assigned tasks: one team member focused on the composite function, the other one on the clamping
- Duration: ~30 minutes

Meeting 2 – Review & Integration

- Shared individual implementations
- Tested composite function with sample images
- Adjusted clamping logic to handle edge cases
- Duration: ~30 minutes

Records

- TODO: KakaoTalk screenshots
- Example: discussion on how to handle overflow/underflow

Member's Thoughts on This Project

Lessons Learned

- Even small programs benefit from modular design

- Pixel-level operations require attention to overflow/underflow
- Collaboration and code review help catch subtle mistakes

Future Improvements

- Support for blending with weights (alpha blending)
- Adding more composite modes (multiply, divide, etc.)